

LLD Case Study 7: Ride Sharing Application (Uber / Lyft)

Target Patterns: Strategy Pattern, Observer Pattern, State Pattern, Pricing Calculations

Key Engineering Focus: Driver-rider matching heuristics, dynamic surge fare calculation, spatial coordinates, and trip lifecycle states.

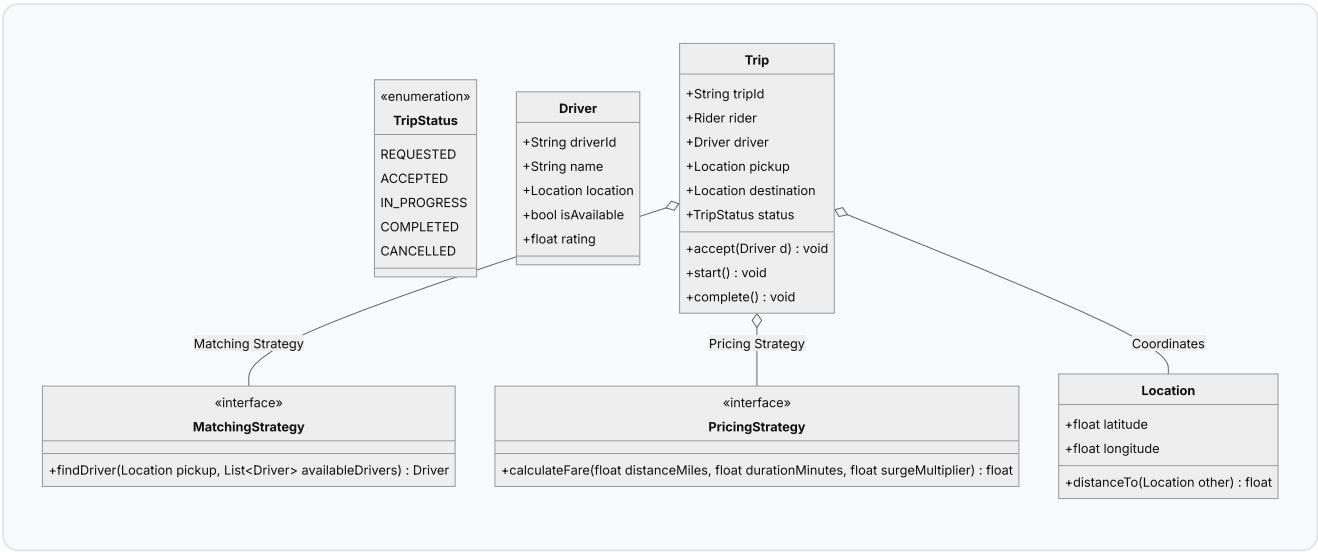
1. Problem Statement & Functional Requirements

Design the core matching and trip dispatch engine for an on-demand ride sharing service.

Requirements:

- 1. Driver Matching Policy (Strategy Pattern):** Pluggable driver matching (e.g. Nearest Driver vs Top-Rated Driver).
 - 2. Fare Calculation Engine (Strategy Pattern):** Dynamic fare calculation combining base fare, distance traveled, trip duration, and real-time surge multiplier.
 - 3. Trip State Machine (State Pattern):** Trip progresses through REQUESTED → ACCEPTED → DRIVER_ARRIVED → IN_PROGRESS → COMPLETED .
 - 4. Real-Time Notification (Observer Pattern):** Broadcast state changes to Rider and Driver interfaces instantly.
-

2. Architecture & UML Class Diagram



3. Production-Grade Python Implementation

